PERSONAL REFERENCE

Linux Command Notes

Organised notes from hands-on command-line practice: searching, sorting, finding, cutting, editing and summarising text, plus processes and permissions. Every command here was run on bash with GNU coreutils.

01 Terminal basics
02 grep: search inside files
03 sort and uniq
04 find: locate files
05 cut: extract columns
06 sed: stream editor
07 head and tail
08 awk: fields, conditions, totals
09 Processes
10 Permissions
11 Pipeline recipes
12 Mistakes I made, and the fixes

01 Terminal basics

TIP	HOW
Auto-list directories	Type `cd` (with a space), then press **Tab** twice
Create a file from the terminal	`cat > file.txt << EOF`, type the lines, finish with `EOF` on its own line
Count lines, number only	`wc -l < file.txt` (plain `wc -l file.txt` also prints the file name)
Run a command in the background	`command &`

```
cat > students.txt << EOF
John Passed
Mary Failed
EOF
```

> The closing word must match exactly: `EOF`, not `eof`. Until it matches, the terminal keeps showing the `>` prompt and waits for more input.

02 grep: search inside files

COMMAND	WHAT IT DOES
`grep "word" file`	Print lines that contain *word*
`grep -i "word" file`	Ignore upper/lower case
`grep -c "word" file`	Count matching lines (same as `grep "word" file | wc -l`)
`grep -n "word" file`	Show line numbers
`grep -ic`, `grep -in`	Options can be combined
`grep -E "apple|banana" file`	Match either word

> Everything after the pattern is treated as a file name: `grep apple banana fruits.txt` looks for a file called *banana*.

03 sort and uniq

COMMAND	WHAT IT DOES
`sort file`	Sort alphabetically
`sort -n file`	Sort numerically
`sort -r` / `sort -nr`	Reverse / largest number first
`sort -u file`	Sort and drop duplicates in one step
`sort -k2,2n file`	Sort by column 2 only, numerically
`uniq file`	Remove **adjacent** duplicates only
`sort file | uniq`	Remove duplicates throughout the file
`sort file | uniq -c`	Count each value (uniq needs sorted input)

Most repeated item:

```
sort fruits.txt | uniq -c | sort -nr | head -n 1                    # top line only
sort fruits.txt | uniq -c | sort -nr | awk 'NR==1 {max=$1} $1==max'   # keeps ties
```

How the awk part works:

- `NR==1 {max=$1}` : on line 1 (the highest count, after `sort -nr`), store column 1 in `max`.
- `$1==max` : print every line whose count equals `max`. A condition with no action prints the line.
- `head -n 1` shows only one line, so if two items are tied for the top count, one is silently hidden.

04 find: locate files

COMMAND	WHAT IT DOES
`find ./testdir -name "*.txt"`	All `.txt` files, including subdirectories
`find ./testdir -type f`	Files only
`find ./testdir -type d`	Directories only
`find ./testdir -type f | wc -l`	Count all files, whatever the extension
`find ./testdir \( -name "*.txt" -o -name "*.log" \)`	Match several patterns

> Keep the pattern in quotes (`"*.txt"`) so the shell passes it to find unexpanded. Brace patterns like `"{.txt,.log}"` do not work in `-name` .

05 cut: extract columns

```
101:John:Developer
102:Mary:Tester
103:Alex:DevOps
```

```
cut -d ':' -f1 employees.txt    # IDs:   101 102 103
cut -d ':' -f2 employees.txt    # names: John Mary Alex
cut -d ':' -f3 employees.txt    # roles: Developer Tester DevOps
```

- `cut` extracts columns from text.
- `-d ':'` uses `:` as the delimiter.
- `-f2` selects field (column) 2; `-f1,3` selects several.

06 sed: stream editor

COMMAND	WHAT IT DOES
`sed 's/linux/unix/' file`	Replace the **first** match on each line (output only)
`sed 's/linux/unix/g' file`	Replace **every** match on each line
`sed '2s/old/new/' file`	Replace only on line 2
`sed -i 's/linux/ubuntu/g' file`	Save the change into the file
`sed -n '2p' file`	Print only line 2
`sed '1d' file`	Display the file without line 1

- `-n` tells sed not to print anything automatically. Without it, `sed '2p'` prints every line and line 2 a second time.
- sed is case-sensitive: `s/linux/.../` will not touch `Linux`.

07 head and tail

COMMAND	WHAT IT DOES
`head -n 1 file`	First line
`head -n -1 file`	Every line except the last
`tail -n 3 file`	Last 3 lines

08 awk: fields, conditions, totals

`awk 'condition {action}' file` runs once for every line. `$1`, `$2` are fields (columns), `$0` is the whole line, `NR` is the current line number.

`awk 'condition'` is effectively the same as `awk 'condition {print $0}'`.

COMMAND	WHAT IT DOES
`awk '{print}' file`	Print all lines
`awk 'NR==1' file`	Print line 1
`awk 'NR==2' file`	Print line 2
`awk 'NR>2' file`	Print the lines after line 2
`awk 'NR!=1' file`	Print all lines except line 1
`awk '{print $1}' file`	Print the first field
`awk -F ':' '{print $2}' file`	Print the second colon-separated field
`awk '$2>80 {print $1}' marks.txt`	Names of students scoring above 80

Totals and averages:

```
awk '{sum += $2; print sum}' marks.txt          # running total: 85 177 247 342
awk '{sum += $2} END {print sum}' marks.txt     # total only:    342
awk '{sum += $2} END {print sum/NR}' sales.txt  # average:       6000
```

`END { }` runs once, after the last line. Inside it, `NR` holds the total number of lines.

09 Processes

COMMAND	WHAT IT DOES
`sleep 500 &`	Start a process in the background
`ps -u | grep sleep`	List my processes containing *sleep* (also lists the grep itself)
`pgrep -a sleep`	PIDs and command lines, without matching itself
`pgrep -c sleep`	Count them
`kill PID`	Stop one process
`killall sleep` or `pkill sleep`	Stop every process named *sleep* in one step

> `kill` does not read PIDs from a pipe. To feed it a list: `kill $(pgrep sleep)` .

10 Permissions

Each octal digit is owner / group / others, built from **r = 4, w = 2, x = 1** (so 7 = rwx, 5 = r-x, 4 = r--, 0 = none).

COMMAND	RESULT
`chmod 700 secret.txt`	Owner rwx, group and others nothing
`chmod a+r secret.txt`	Everyone can read
`chmod go-w secret.txt`	Remove write for group and others
`chmod u+x,go-x secret.txt`	Executable only for the owner

Symbolic form: who (`u` owner, `g` group, `o` others, `a` all), then `+` add, `-` remove or `=` set exactly, then the permission letters.

11 Pipeline recipes

GOAL	COMMAND
Count each value	`sort names.txt | uniq -c`
Most repeated value (ties kept)	`sort names.txt | uniq -c | sort -nr | awk 'NR==1 {max=$1} $1==max'`
Only the most repeated name	`sort names.txt | uniq -c | sort -nr | head -n 1 | awk '{print $2}'`
Highest value in column 2	`sort -k2,2nr sales.txt | awk 'NR==1 {max=$2} $2==max'`
Lowest value in column 2	`sort -k2,2n sales.txt | awk 'NR==1 {min=$2} $2==min'`
Total of column 2	`awk '{sum += $2} END {print sum}' sales.txt`
Average of column 2	`awk '{sum += $2} END {print sum/NR}' sales.txt`

GOAL	COMMAND
Top 3 by column 2	`sort -k2,2nr sales.txt | head -n 3`

12 Mistakes I made, and the fixes

WHAT I TYPED	WHAT HAPPENED	FIX
`uniq fruits.txt`	Duplicates still there	`sort fruits.txt | uniq` (uniq only removes adjacent repeats)
`sort names.txt | uniq -c | head -n 1`	Printed *alex* because it sorts first, not because it is most frequent	Add `sort -nr` before `head`
`ps -u | grep -ic sleep`	Counted 4 instead of 3	The grep matched itself; use `pgrep -c sleep`
`ps -u | grep sleep | kill`	kill usage error	`kill $(pgrep sleep)` or `killall sleep`
`sed '2p' file`	Every line printed, line 2 twice	`sed -n '2p' file`
`sed 's/linux/unix/' file`	Only the first match per line replaced	Add `g` : `s/linux/unix/g`
`sort -n sales.txt`	Sorted by name, not amount	`sort -k2,2n sales.txt`
`sort -k2, 2nr`	"invalid number" error	No space: `-k2,2nr`
`awk '{sum += $2; print sum}'`	Running total on every line	Print in `END`
`awk '{sum+=%2}'`	Syntax error	Fields are `$2`
`awk '... 1==$max'`	Printed nothing	Compare `$1==max` (`$max` means "field number max")
`find ./testdir -name "*.logs"`	Found nothing	`"*.log"`
`grep -ic apple banana grape fruits.txt`	"No such file" errors	`grep -E "apple|banana|grape" fruits.txt`
`cut-d ':' -f1`	Command not found	Space after the command: `cut -d ':' -f1`
Closing a heredoc with `eof`	Terminal kept waiting	Must match exactly: `EOF`
`head –n 1` copied from a document	`–n` treated as a file name, not an option	Use a plain hyphen `-n`, not an en dash `–`